

SHIV NADAR UNIVERSITY CHENNAI

Department of Computer Science and Engineering

DEEP LEARNING LABORATORY

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for
Multi-Class Image Classification

Name	: Suresh S
Roll No	: 24011101111
Programme	: B.Tech Computer Science and Engineering
Semester	: V
Course	: Deep Learning Laboratory
Academic Year	: 2026-2027

1 Aim

The objective of this experiment is to implement a Multi-Layer Perceptron (MLP) using TensorFlow/Keras for multi-class image classification on the Fashion-MNIST dataset. The experiment also aims to perform automated hyperparameter optimization using Randomized Search and compare the optimized model with the baseline implementation.

2 Background Theory

Artificial Neural Networks are machine learning models inspired by the biological nervous system. A Multi-Layer Perceptron (MLP) is one of the simplest yet most effective feed-forward neural networks used for supervised learning tasks. Unlike a single-layer perceptron, an MLP contains one or more hidden layers that enable it to learn complex nonlinear relationships between input features and target classes.

Each neuron performs a weighted summation of the inputs followed by an activation function.

$$z = Wx + b$$

where

- W represents the weight matrix,
- x denotes the input vector,
- b is the bias.

The activation of the neuron is computed as

$$a = f(z)$$

where f is a nonlinear activation function such as ReLU, Sigmoid or Tanh.

For multi-class classification, the output layer employs the Softmax activation function

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

which converts the network outputs into probability values whose sum equals one.

Model parameters are optimized using the backpropagation algorithm together with gradient-based optimization techniques such as Adam.

3 MLP Architecture

The baseline neural network implemented in this experiment consists of the following layers.

- Input Layer
 - 784 neurons
 - Flattened 28×28 grayscale image
- Hidden Layer 1
 - 128 neurons
 - ReLU activation
- Hidden Layer 2
 - 64 neurons
 - ReLU activation
- Output Layer
 - 10 neurons
 - Softmax activation

The network is designed to learn meaningful representations of clothing images while maintaining computational efficiency. The hidden layers extract increasingly abstract features from the input images before the Softmax layer performs multi-class classification.

4 Activation Functions

4.1 Rectified Linear Unit (ReLU)

The Rectified Linear Unit is defined as

$$f(x) = \max(0, x)$$

ReLU is widely used because it accelerates convergence, reduces the vanishing gradient problem and is computationally inexpensive.

4.2 Softmax

The Softmax activation function converts raw output scores into probability values.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

It is commonly used for multi-class classification problems since the predicted probabilities sum to one.

5 Loss Function

The network is trained using Sparse Categorical Cross Entropy.

For each training sample,

$$L = -\log(P_{correct})$$

where $P_{correct}$ represents the probability assigned to the correct class. Lower loss values indicate better model performance.

6 Optimizer

The Adam optimizer is used for training because it combines momentum and adaptive learning rates.

Its advantages include

- Faster convergence
- Stable optimization
- Automatic learning-rate adaptation
- Efficient training of deep neural networks

7 Dataset Description

The Fashion-MNIST dataset is a benchmark image classification dataset developed by Zalando Research. It contains grayscale images of clothing items belonging to ten different categories and is commonly used as a replacement for the original handwritten digit MNIST dataset.

Table 1: Dataset Summary

Dataset	Fashion-MNIST
Training Images	60,000
Testing Images	10,000
Image Size	28×28 pixels
Image Type	Grayscale
Number of Classes	10
Input Dimension	784 Features
Task	Multi-Class Classification

The dataset consists of ten clothing categories:

1. T-shirt/Top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle Boot

Before training, all pixel values were normalized to the range $[0, 1]$ by dividing each pixel intensity by 255. Every image was then flattened into a one-dimensional vector containing 784 features so that it could be processed by the fully connected layers of the Multi-Layer Perceptron.

8 Experimental Procedure

The implementation of the Multi-Layer Perceptron was carried out using Python and TensorFlow/Keras in Google Colab. The complete workflow followed during the experiment is summarized below.

1. Import the required libraries including TensorFlow, Keras, NumPy, Pandas, Matplotlib, Seaborn and Scikit-Learn.

2. Load the Fashion-MNIST dataset directly from TensorFlow.
3. Explore the dataset by displaying sample images and observing the class distribution.
4. Normalize the pixel values to the range $[0, 1]$.
5. Flatten every 28×28 image into a one-dimensional vector containing 784 input features.
6. Construct the baseline Multi-Layer Perceptron consisting of two hidden layers with ReLU activation and a Softmax output layer.
7. Compile the model using the Adam optimizer together with Sparse Categorical Cross Entropy loss.
8. Train the network for 20 epochs using a batch size of 32.
9. Evaluate the trained model on the testing dataset.
10. Compute Accuracy, Precision, Recall, F1-score and generate the confusion matrix.
11. Perform automated hyperparameter optimization using Randomized Search with five-fold cross validation.
12. Train the optimized model using the best hyperparameter combination.
13. Compare the baseline and optimized models using various performance metrics and graphical analysis.

9 Source Code

The complete implementation of this experiment, including the notebook and generated plots, is available in the following GitHub repository.

<https://github.com/ssukesh182/DeeplearningLab>

10 Baseline Model Configuration

The architecture used for the baseline model is summarized below.

Table 2: Baseline Model Configuration

Parameter	Value
Input Size	784
Hidden Layer 1	128 Neurons (ReLU)
Hidden Layer 2	64 Neurons (ReLU)
Output Layer	10 Neurons (Softmax)
Optimizer	Adam
Learning Rate	0.001
Loss Function	Sparse Categorical Cross Entropy
Batch Size	32
Epochs	20
Training Time	179.96 seconds

11 Hyperparameter Optimization

To improve the baseline model, automated hyperparameter optimization was performed using Randomized Search together with five-fold cross validation. Instead of manually selecting parameters, several combinations of network architectures and training parameters were evaluated to identify the configuration that produced the best validation performance while reducing computational cost.

The search space used during optimization is shown below.

Table 3: Hyperparameter Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Activation Function	ReLU, Tanh, Sigmoid
Optimizer	SGD, Adam, RMSProp
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Dropout Rate	0.0, 0.2, 0.5

The best combination obtained after Randomized Search is presented below.

Table 4: Best Hyperparameters

Parameter	Best Value
Hidden Layers	1
Hidden Neurons	256
Activation Function	Sigmoid
Optimizer	Adam
Learning Rate	0.001
Batch Size	64
Epochs	10
Dropout	0.0
Cross Validation Accuracy	88.11%

12 Experimental Results

The baseline model achieved good classification performance on the Fashion-MNIST testing dataset. The optimized model achieved comparable accuracy while significantly reducing the training time.

12.1 Baseline Model Performance

Table 5: Baseline Model Performance

Metric	Value
Accuracy	88.85%
Precision	88.93%
Recall	88.85%
F1 Score	88.76%
Training Time	179.96 s

12.2 Optimized Model Performance

Table 6: Optimized Model Performance

Metric	Value
Accuracy	87.67%
Precision	87.67%
Recall	87.67%
F1 Score	87.56%
Training Time	51.80 s

12.3 Baseline vs Optimized Model

Table 7: Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.85%	87.67%
Precision	88.93%	87.67%
Recall	88.85%	87.67%
F1 Score	88.76%	87.56%
Training Time	179.96 s	51.80 s
Optimizer	Adam	Adam
Learning Rate	0.001	0.001
Batch Size	32	64
Activation Function	ReLU	Sigmoid
Hidden Layers	2	1

The baseline model achieved the highest testing accuracy of 88.85%, whereas the optimized model achieved a slightly lower accuracy while reducing the training time by more than 70%. The optimization process selected a simpler network architecture with fewer hidden layers, demonstrating that comparable performance can often be achieved with lower computational complexity.

13 Result Analysis

13.1 Sample Fashion-MNIST Images

10 Sample Fashion-MNIST Images

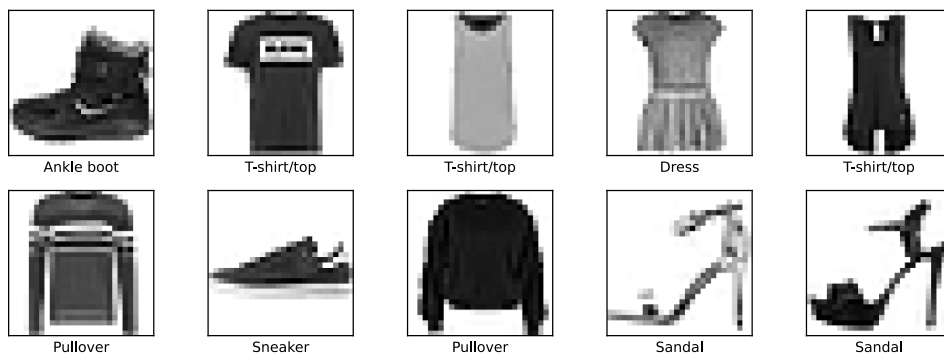


Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference:

The figure shows representative samples from all clothing categories available in the Fashion-MNIST dataset. Although the images are only 28×28 grayscale pixels, they still preserve enough visual information for effective classification by the neural network.

13.2 Class Distribution

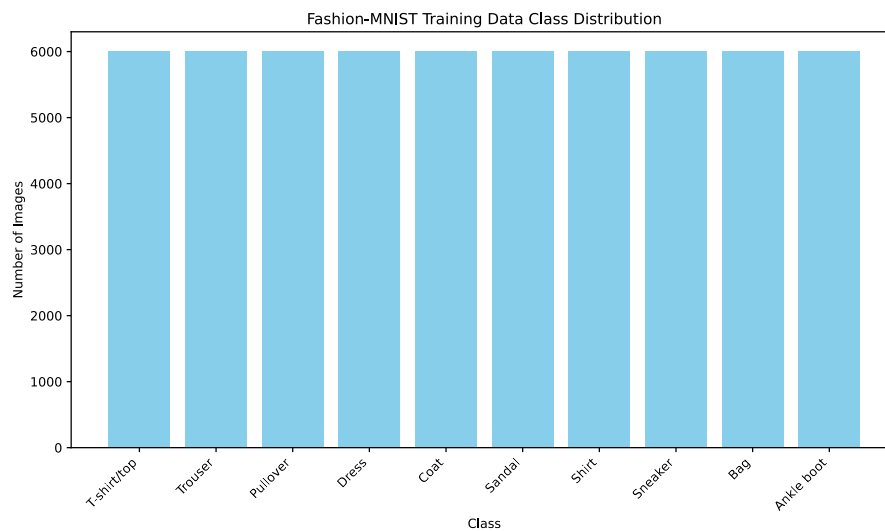


Figure 2: Distribution of Classes in the Dataset

Inference:

The dataset is evenly balanced across all ten classes, with each category containing nearly the same number of samples. This balanced distribution minimizes bias during training and allows the model to learn each class equally well.

13.3 Baseline Model Confusion Matrix

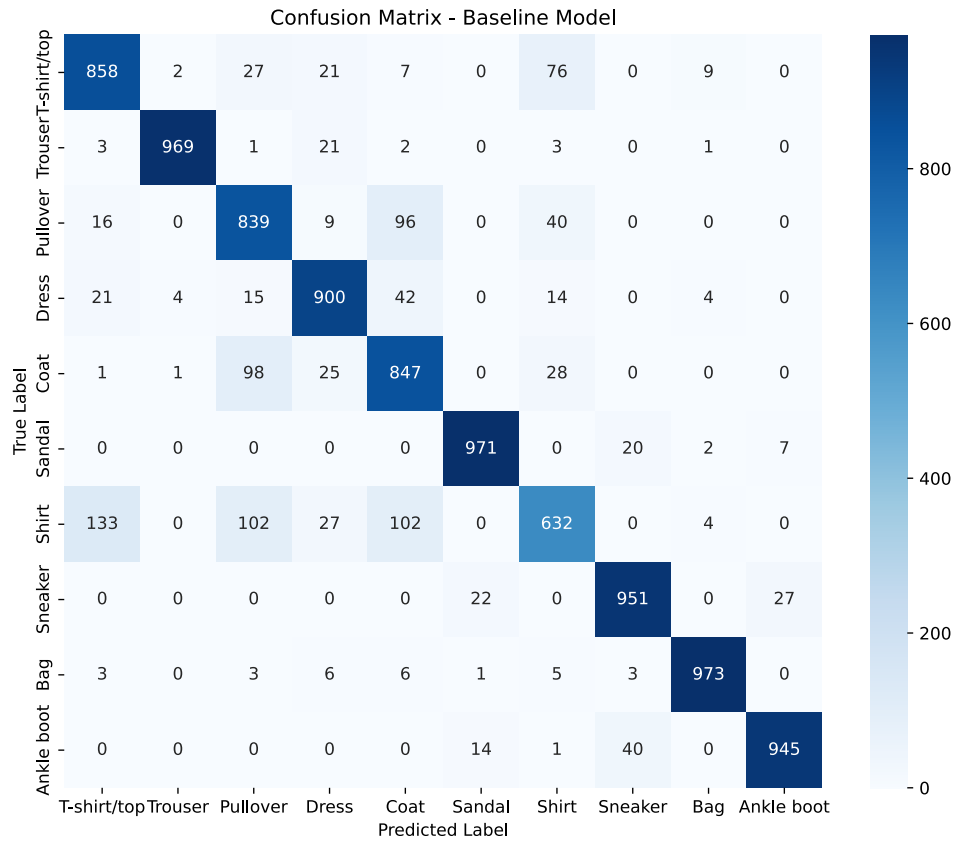


Figure 3: Confusion Matrix of the Baseline Model

Inference:

Most samples lie along the diagonal, indicating correct predictions for the majority of test images. A few misclassifications occur between visually similar clothing categories such as shirts, pullovers and coats, which is expected because these classes share similar appearance.

13.4 Baseline Model Accuracy

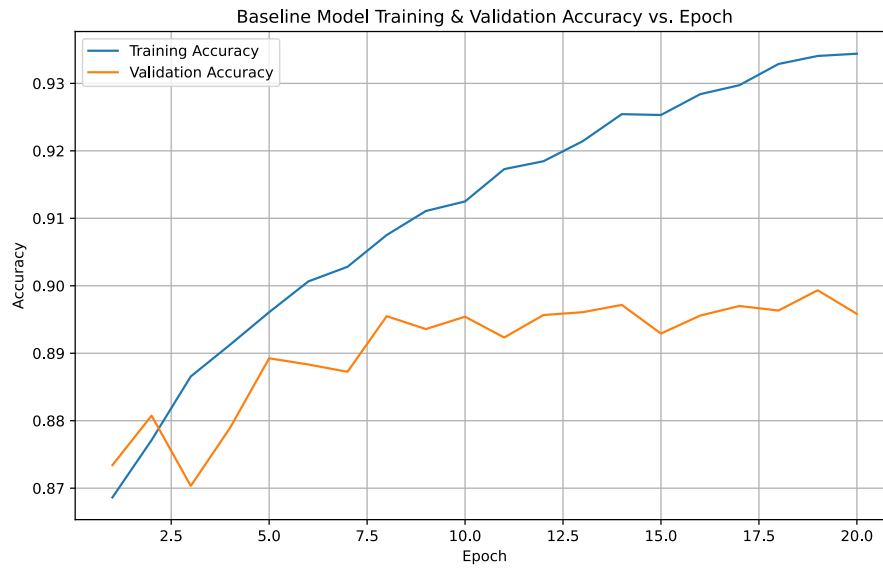


Figure 4: Baseline Model Accuracy During Training

Inference:

Training accuracy steadily increases with each epoch before gradually converging. The smooth learning curve indicates stable optimization and demonstrates that the model successfully learns useful image features throughout training.

13.5 Baseline Model Loss

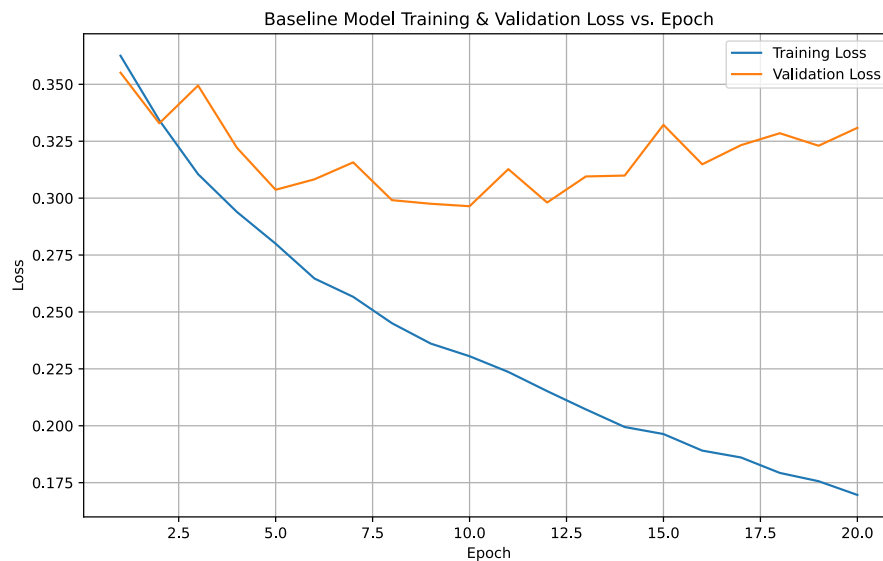


Figure 5: Baseline Model Loss During Training

Inference:

The training loss decreases consistently as the epochs progress. The absence of sudden fluctuations indicates stable convergence and confirms that the optimization process effectively minimizes classification error.

13.6 Optimized Model Confusion Matrix

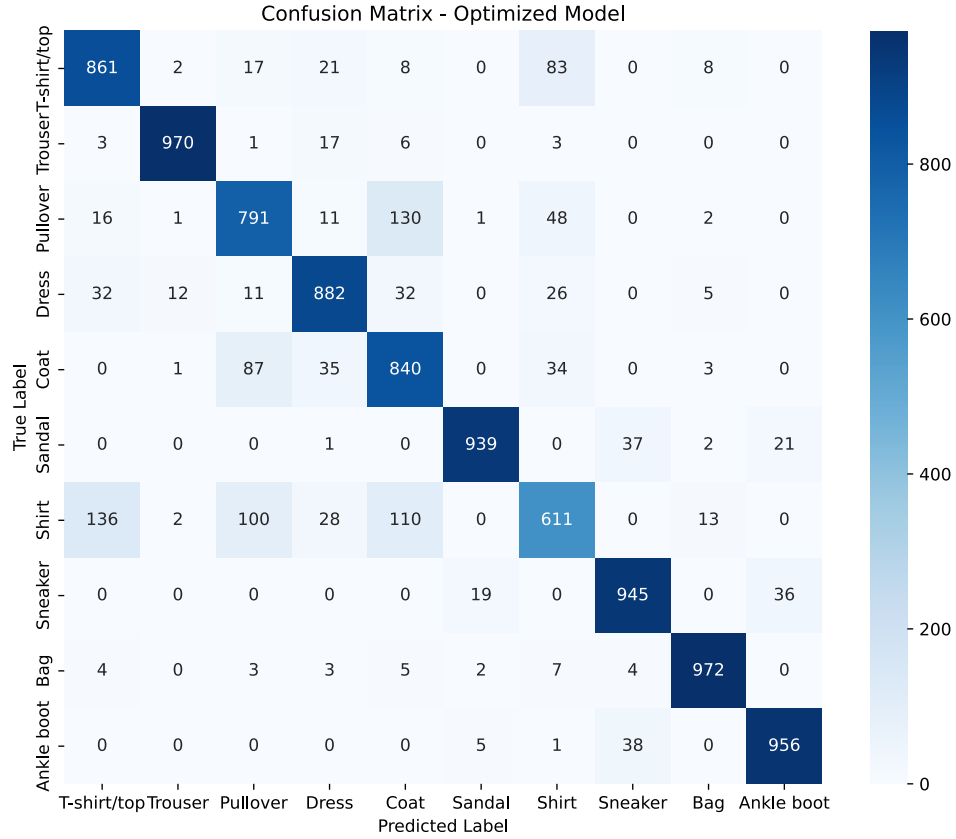


Figure 6: Confusion Matrix of the Optimized Model

Inference:

The optimized model also produces a strong diagonal pattern, indicating good classification accuracy. Although its overall accuracy is slightly lower than the baseline model, it maintains reliable predictions while using a simpler architecture.

13.7 Optimized Model Accuracy

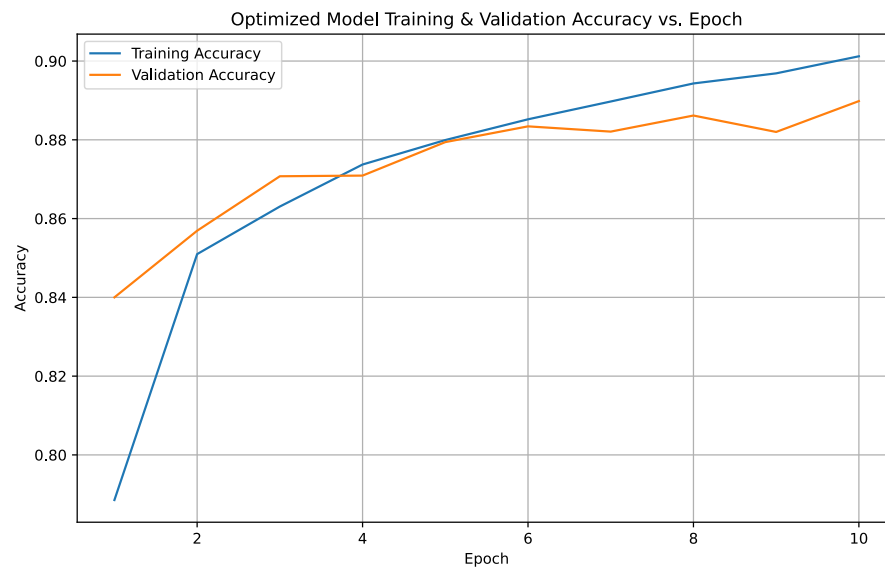


Figure 7: Optimized Model Accuracy During Training

Inference:

The optimized model reaches a high accuracy within fewer training epochs. This demonstrates that the selected hyperparameters improve learning efficiency and reduce the amount of computation required.

13.8 Optimized Model Loss

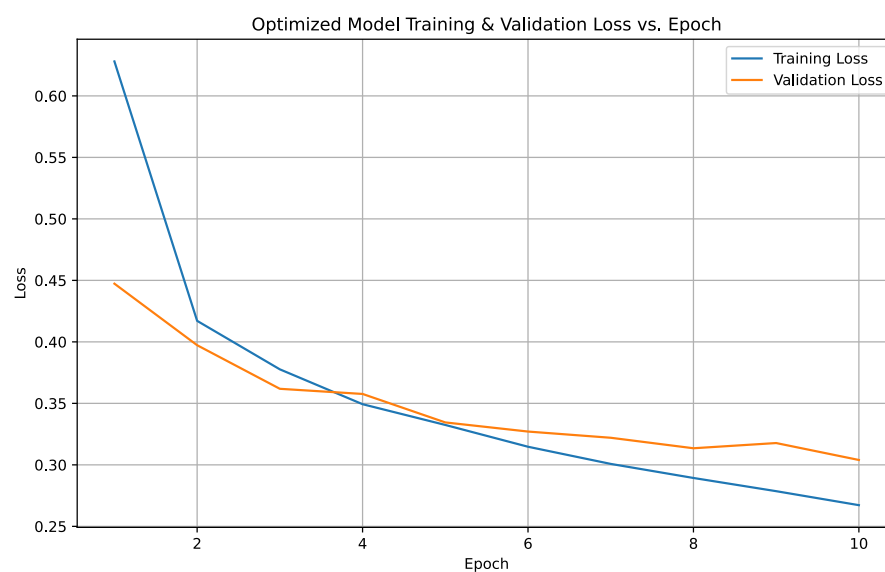


Figure 8: Optimized Model Loss During Training

Inference:

The loss decreases rapidly during the initial epochs before stabilizing. The learning process remains smooth throughout training, indicating that the optimized configuration converges without instability.

13.9 Hyperparameter Search Results

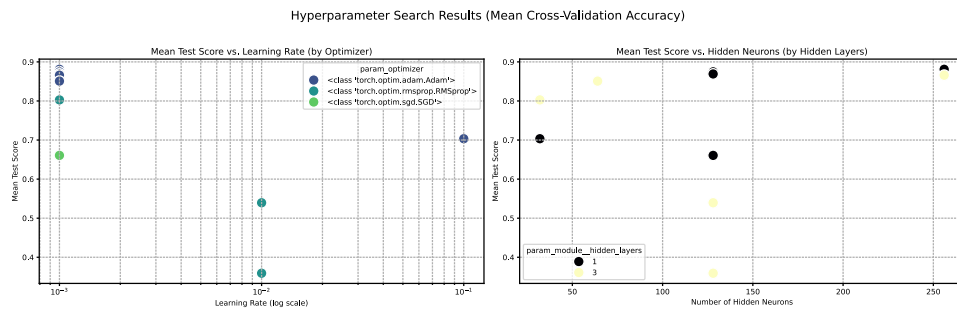


Figure 9: Performance of Different Hyperparameter Combinations

Inference:

The Randomized Search evaluates multiple combinations of hyperparameters and identifies the configuration producing the highest validation accuracy. This automated approach removes the need for manual parameter tuning and improves the efficiency of model development.

13.10 Model Accuracy Comparison

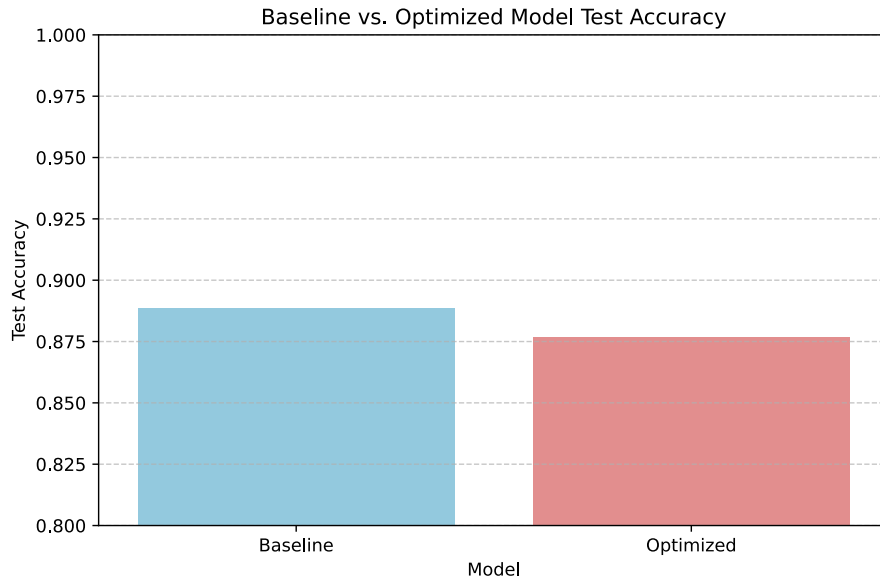


Figure 10: Comparison of Baseline and Optimized Model Accuracy

Inference:

The baseline model achieves the highest testing accuracy of 88.85%, while the optimized model achieves a very similar accuracy with significantly reduced training time. This demonstrates that a simpler network can often provide competitive performance with lower computational cost.

14 Discussion

The experiment successfully demonstrated the implementation of a Multi-Layer Perceptron for multi-class image classification using TensorFlow/Keras. The baseline model achieved an accuracy of 88.85%, indicating that fully connected neural networks are capable of extracting meaningful features from grayscale clothing images.

Hyperparameter optimization using Randomized Search selected a simpler architecture consisting of a single hidden layer with 256 neurons and Sigmoid activation. Although the optimized model achieved a slightly lower testing accuracy of 87.67%, its training time was reduced from approximately 180 seconds to about 52 seconds. This shows that model optimization is not always focused on maximizing accuracy alone but also aims to improve computational efficiency.

The confusion matrices reveal that most classification errors occur between visually similar clothing categories such as shirts, coats and pullovers. Overall, both models demonstrate strong generalization ability on unseen testing data.

15 Conclusion

In this experiment, a Multi-Layer Perceptron was successfully developed for Fashion-MNIST image classification using TensorFlow/Keras. The baseline model achieved an accuracy of 88.85%, while the optimized model obtained comparable performance with substantially lower computational cost after automated hyperparameter tuning.

The experiment provided practical experience in neural network construction, image preprocessing, model evaluation and hyperparameter optimization. It also demonstrated the trade-off between predictive accuracy and computational efficiency, highlighting the importance of selecting suitable model parameters for real-world deep learning applications.

16 XOR: Perceptron Limitation and MLP Solution

16.1 Theory

The XOR (exclusive-OR) function outputs 1 only when its two inputs differ, giving the truth table $\{(0,0) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1, (1,1) \rightarrow 0\}$. Unlike AND, OR, and NOT, the positive and negative classes of XOR cannot be separated by any single straight line in the input plane – the class-1 points $(0,1)$ and $(1,0)$ lie diagonally opposite the class-0 points $(0,0)$ and $(1,1)$. XOR is therefore *not linearly separable*, and by the perceptron convergence theorem, a single-layer perceptron with a linear decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ can never learn it, no matter how many epochs it is trained for. Instead of converging, the weights enter a repeating cycle of updates.

This limitation is overcome by a **Multi-Layer Perceptron (MLP)**, which adds one or more hidden layers between the input and output. Each hidden neuron applies a non-linear activation (here, the sigmoid $\sigma(x) = 1/(1 + e^{-x})$) to a linear combination of the inputs, and the outputs of these hidden neurons are then recombined at the output layer. This composition of non-linear transformations lets the network carve out a non-linear (curved) decision region, which is sufficient to separate XOR's classes. The MLP is trained using **backpropagation**: the output error is propagated backward through the network via the chain rule, and each weight matrix is updated using gradient descent,

$$W \leftarrow W + \eta \delta^\top a, \quad b \leftarrow b + \eta \sum \delta \quad (1)$$

where δ is the local error term at a given layer and a is the corresponding layer's input activation.

16.2 Single-Layer Perceptron on XOR

A single-layer perceptron ($\mathbf{w} = [w_1, w_2]$, b , step activation, $\eta = 1$) was trained on the XOR truth table for a maximum of 10 epochs (4 samples/epoch). Unlike AND/OR/NOT, training *never* reaches an error-free epoch – every epoch produces exactly 3 or 4 updates, and the weight values repeat in a fixed cycle from Update 7 onward. Table 8 shows the first two epochs, where the cyclic pattern already emerges; this same pattern (Updates 7–10, 11–14, $\dots$, 35–38) repeats identically for every remaining epoch up to the maximum of 10, so training stops at $\mathbf{w} = [-1, 0]$, $b = 0$ – a boundary that misclassifies $(0, 1)$ and $(1, 0)$, i.e. it never solves XOR.

Table 8: Weight updates during perceptron training on XOR (Epochs 1–2; the same 4-update cycle then repeats for Epochs 3–10)

Update	Epoch	Input (x_1, x_2)	Target	Predicted	Error	Updated $\mathbf{w}$	Updated b
1	1	(0,0)	0	1	−1	(0.0, 0.0)	−1
2	1	(0,1)	1	0	+1	(0.0, 1.0)	0
3	1	(1,1)	0	1	−1	(−1.0, 0.0)	−1
4	2	(0,1)	1	0	+1	(−1.0, 1.0)	0
5	2	(1,0)	1	0	+1	(0.0, 1.0)	1
6	2	(1,1)	0	1	−1	(−1.0, 0.0)	0

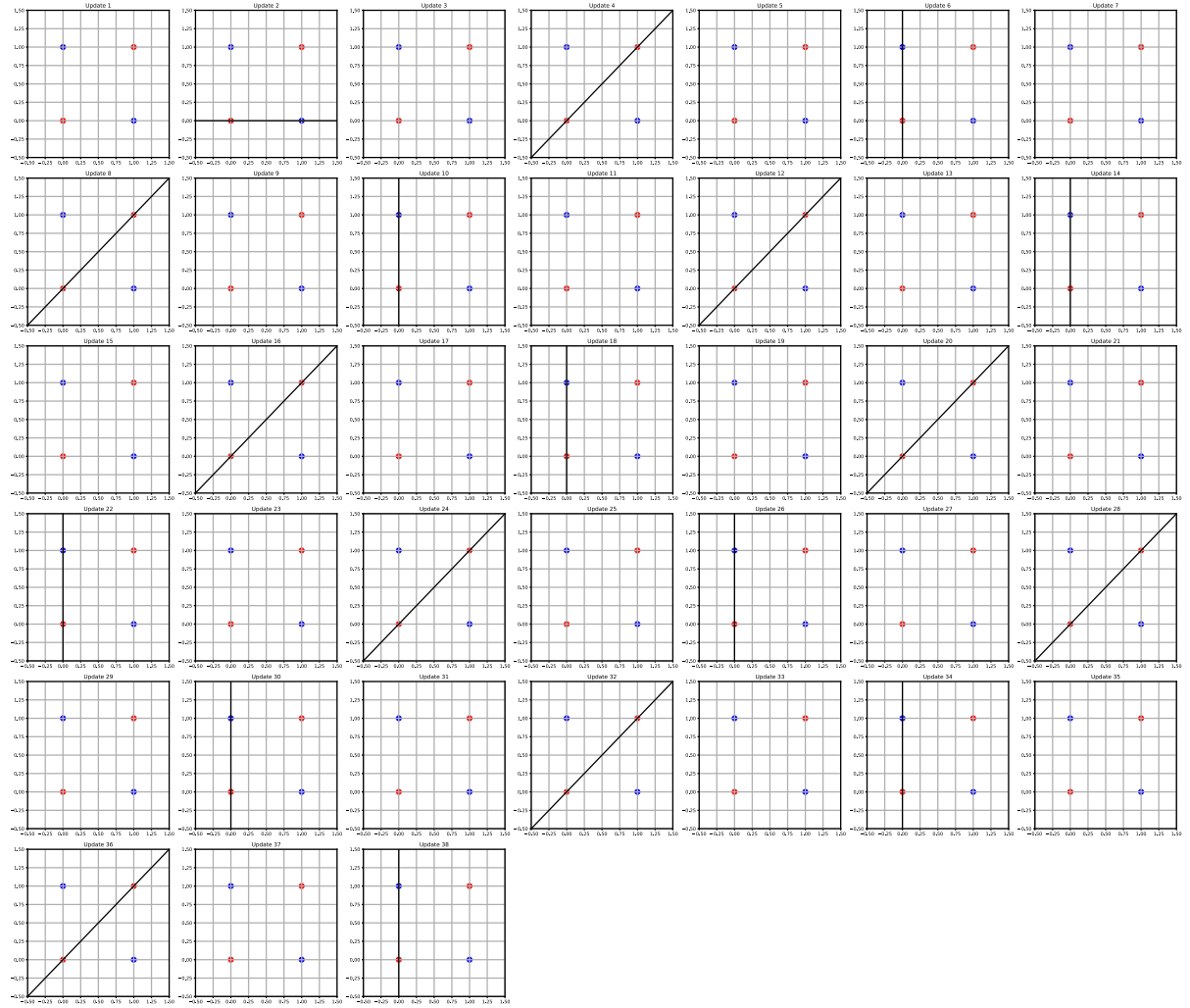


Figure 11: All 38 weight updates of the single-layer perceptron on XOR across 10 epochs; the boundary keeps oscillating and never separates the classes

16.3 Multi-Layer Perceptron on XOR

An MLP with a 2-4-1 architecture (2 input neurons, 4 hidden neurons with sigmoid activation, 1 output neuron with sigmoid activation) was trained with backpropagation for 10,000 epochs at a learning rate of $\eta = 0.1$. The mean squared error loss dropped steadily and had nearly converged by the end of training, as summarized in Table 9.

Table 9: MLP training loss (MSE) at selected epochs

Epoch	Loss (MSE)
1000	0.2389
2000	0.1855
3000	0.0945
4000	0.0263
5000	0.0116
6000	0.0069
7000	0.0047
8000	0.0036
9000	0.0028
10000	0.0023

Table 10 shows the final predictions on all four XOR input combinations. The raw sigmoid outputs are close to 0 or 1, and thresholding at 0.5 recovers the exact XOR truth table.

Table 10: Trained MLP predictions on the XOR truth table

x_1	x_2	Expected Output	Predicted Output (raw)	Predicted Output (binary)
0	0	0	0.0524	0
0	1	1	0.9536	1
1	0	1	0.9537	1
1	1	0	0.0476	0

16.4 Observations

The single-layer perceptron cannot solve XOR: rather than converging, its weights enter a repeating 3–4 update cycle every epoch, confirming that no straight-line boundary can separate the classes. The MLP, by introducing a hidden layer with a non-linear sigmoid activation, learns a curved decision surface (visible as the diagonal band in Figure 12) that correctly isolates the two diagonal input pairs, successfully solving XOR with near-zero final loss.

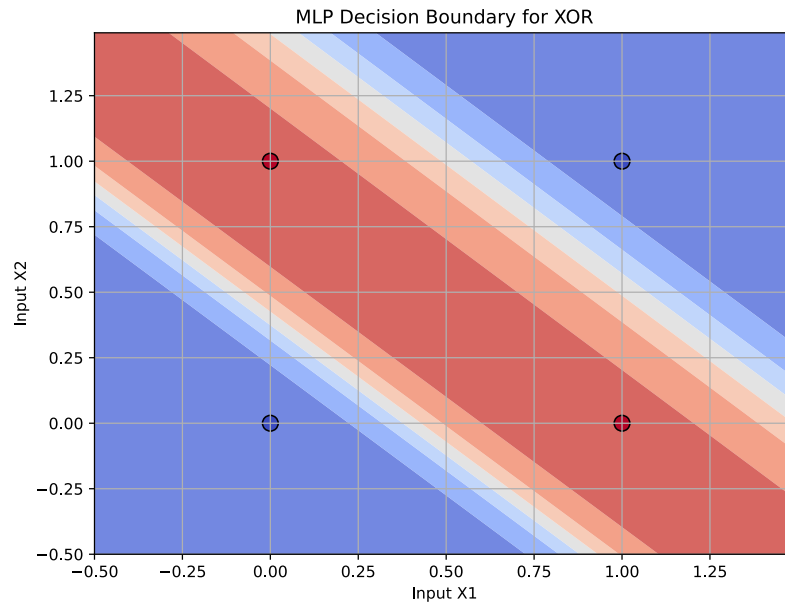


Figure 12: Non-linear decision boundary learned by the MLP for XOR; the red region correctly encloses the class-1 points (0, 1) and (1, 0) while the blue region encloses class-0 points (0, 0) and (1, 1)

17 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press.
2. TensorFlow Documentation.
<https://www.tensorflow.org>
3. Keras Documentation.
<https://keras.io>
4. Fashion-MNIST Dataset.
<https://github.com/zalando-research/fashion-mnist>
5. Scikit-Learn Documentation.
<https://scikit-learn.org>